

Experiment 2: Case Study

Food Delivery Management System — An Original Case Study

Name: Pardheev Nalla RollNo:2420090145

Problem Statement: Create an original real-world case study based on an application of your choice. The case study should include an introduction, user stories, a chosen software development process model, suitable programming language and technologies, an implementation strategy, an evaluation strategy, at least two UML diagrams, and a conclusion.

Real-World Case Study: Food Delivery Management System (FDMS) — a platform connecting customers, restaurants, and delivery agents for ordering and real-time food delivery.

1. Introduction

What is a Food Delivery Management System?

A Food Delivery Management System (FDMS) is a software platform that connects three parties — customers, restaurants, and delivery agents — to enable food ordering, order preparation tracking, and real-time delivery. It replaces phone-based ordering with a searchable app, live order tracking, and integrated online payments.

Core idea:

One platform, three connected roles, real-time order and delivery tracking from checkout to doorstep.

Scope of the System

- Restaurant and menu browsing
- Order placement and online payment
- Restaurant-side order acceptance and preparation tracking
- Delivery agent assignment and real-time location tracking
- Ratings, reviews, and order history
- Sales and performance reporting for restaurants

2. User Stories

A User Story is a short, simple description of a software feature written from the end user's perspective, in the format: "As a [user], I want [functionality] so that [benefit]."

Customer

User ID	User Story	Priority
US-01	As a customer, I want to browse nearby restaurants so that I can decide where to order food from.	High
US-02	As a customer, I want to search a restaurant's menu so that I can quickly find dishes I like.	High
US-03	As a customer, I want to place an order with selected items so that my food gets prepared and delivered.	High
US-04	As a customer, I want to track my order in real time so that I know when it will arrive.	High
US-05	As a customer, I want to pay online securely so that I don't have to handle cash on delivery.	High
US-06	As a customer, I want to rate and review my order so that I can share feedback with other users.	Low
US-07	As a customer, I want to view my past orders so that I can reorder my favorites easily.	Medium

Delivery Agent

User ID	User Story	Priority
US-08	As a delivery agent, I want to accept delivery requests so that I can start earning for the trip.	High
US-09	As a delivery agent, I want to update delivery status so that customers and the restaurant know the order's progress.	High
US-10	As a delivery agent, I want turn-by-turn navigation to the restaurant and customer so that I can deliver orders efficiently.	Medium
US-11	As a delivery agent, I want to view my daily and weekly earnings so that I can track my income.	Medium

Restaurant Admin

User ID	User Story	Priority
US-12	As a restaurant admin, I want to manage my menu and item availability so that customers see accurate offerings.	High
US-13	As a restaurant admin, I want to accept or reject incoming orders so that I only commit to orders I can fulfil.	High

User ID	User Story	Priority
US-14	As a restaurant admin, I want to update order preparation status so that the app and delivery agent stay informed.	High
US-15	As a restaurant admin, I want to view sales reports so that I can track revenue and popular items.	Medium

3. Software Development Process Model

Rapid Application Development (RAD)

Reason for Selection

- Food delivery platforms compete heavily on time-to-market, so RAD's focus on fast prototyping suits the need to launch quickly and iterate.
- Core user flows (browsing, ordering, tracking) can be prototyped early and demoed to real users for fast feedback.
- Heavy reliance on UI/UX (maps, live tracking, order status) benefits from RAD's iterative, user-driven refinement rather than a long upfront design phase.
- Third-party components (maps API, payment gateway, notifications) can be integrated and tested incrementally within each prototype cycle.
- Business rules (delivery radius, commission, surge pricing) often change post-launch, which RAD's short cycles can absorb better than a heavily sequential model.

RAD Cycle Plan

Cycle	Focus
Cycle 1	Business Modeling — define customer, restaurant, and delivery agent workflows
Cycle 2	Data Modeling — define orders, menus, users, and delivery entities
Cycle 3	Process Modeling — prototype ordering, payment, and assignment logic
Cycle 4	Application Generation — build working prototypes for each role's app
Cycle 5	Testing & Turnover — user testing, refinement, and deployment

4. Programming Language and Technologies

Backend

Node.js (Express.js)

- Non-blocking, event-driven architecture suited to real-time order and location updates
- Fast development cycle, aligning well with the RAD process model
- Large ecosystem for payment, maps, and notification integrations
- Easy to scale horizontally for high order volumes during peak hours

Frontend / Mobile

- React Native (Customer and Delivery Agent mobile apps)
- React.js (Restaurant Admin web dashboard)

Database

- MongoDB (flexible schema for menus, orders, and order-status history)

Other Technologies

- Socket.io (real-time order status and live delivery tracking)
- Google Maps API (geolocation, route optimization, distance-based delivery fee)
- Razorpay / Stripe (payment gateway integration)
- Redis (caching restaurant/menu data and managing agent-assignment queues)
- Firebase Cloud Messaging (push notifications)
- Docker, Git/GitHub, AWS (deployment and hosting)

5. Implementation Strategy

Phase 1 – Requirement Analysis

- Gather requirements from customers, restaurant partners, and delivery agents.
- Prepare the Software Requirement Specification (SRS).
- Identify core entities: Customer, Restaurant, MenuItem, Order, DeliveryAgent, Payment.

Phase 2 – System Design

- Design the database schema for orders, menus, and delivery assignments.
- Design UI wireframes/prototypes for the customer app, agent app, and restaurant dashboard.
- Prepare UML diagrams (use case and sequence diagrams).

Phase 3 – Development (per RAD cycle)

- Build restaurant browsing, menu search, and cart functionality.
- Build order placement and payment integration.
- Build restaurant order-acceptance and status-update flow.
- Build delivery agent assignment and real-time tracking.
- Build ratings, order history, and sales-reporting features.

Phase 4 – Testing

Unit Testing

- Menu search and filtering
- Delivery fee and ETA calculation

Integration Testing

- Order placement + Payment gateway
- Order assignment + Live tracking

System Testing

- End-to-end order-to-delivery workflow testing

User Acceptance Testing (UAT)

- Pilot testing with real customers, a partner restaurant, and delivery agents.

Phase 5 – Deployment

- Docker Container
- AWS EC2 / Elastic Load Balancer
- MongoDB Atlas (managed database)
- CDN for static assets and app updates

Phase 6 – Maintenance

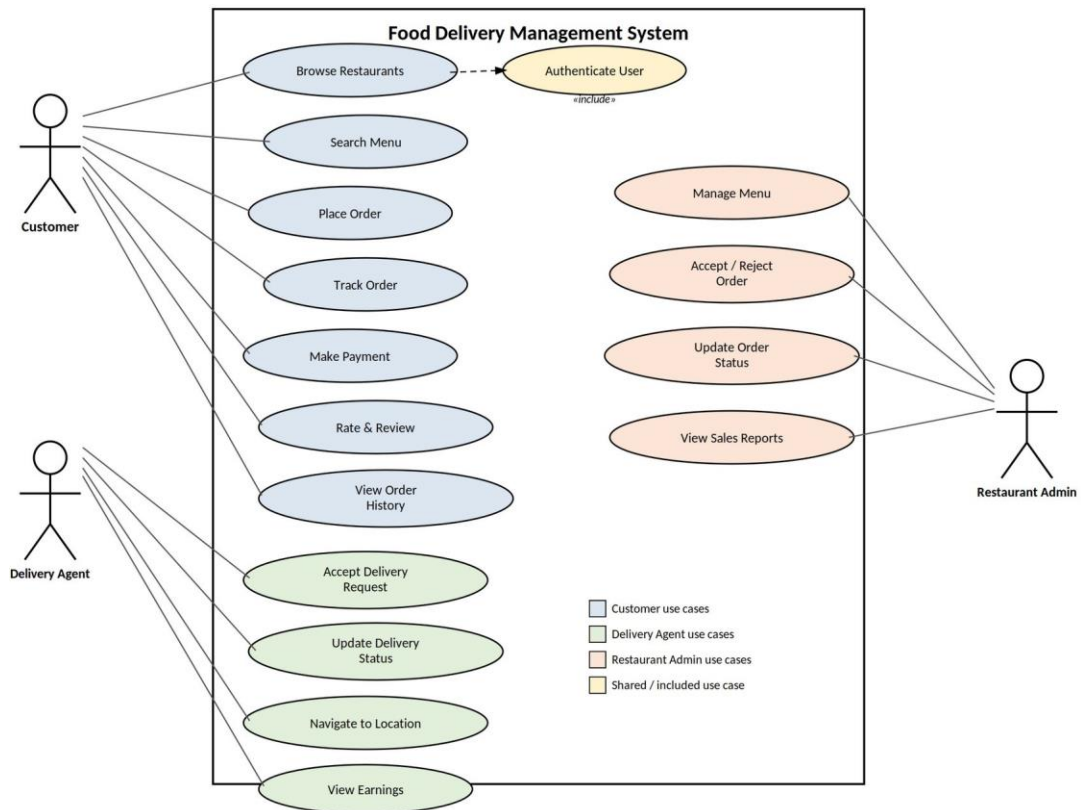
- Monitor live order volumes, fix defects, and tune delivery-assignment logic as demand patterns evolve.

6. Evaluation Strategy

Evaluation Parameter	Method
Functional Accuracy	Requirement verification against user stories
Performance	Load testing peak-hour order volume using JMeter
Security	Authentication, payment data protection, and penetration testing
Reliability	Continuous monitoring of order and tracking services, backup and recovery
Usability	Feedback survey from customers, restaurant staff, and delivery agents
Scalability	Concurrent order and multi-city load testing
Maintainability	Code review and API documentation quality checks

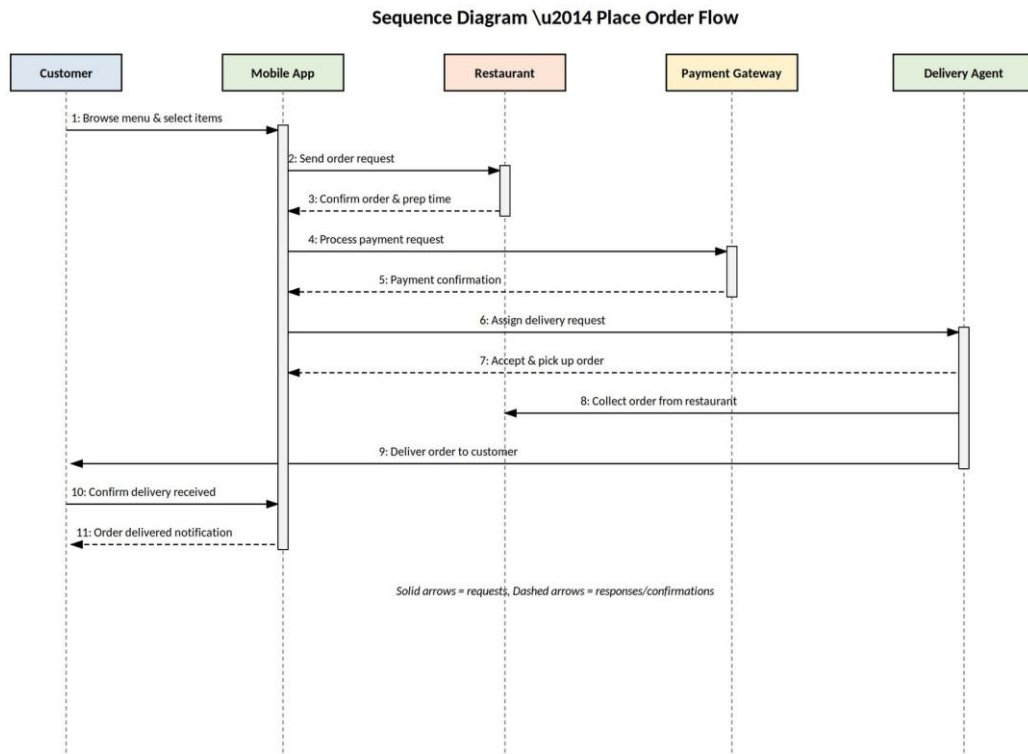
7. UML Diagrams

7.1 Use Case Diagram



The diagram shows the three actors of the Food Delivery Management System — Customer, Delivery Agent, and Restaurant Admin — and the use cases each interacts with. “Authenticate User” is a shared use case included by role-specific actions such as “Browse Restaurants”.

7.2 Sequence Diagram — Place Order Flow



The sequence diagram traces a single order from placement to delivery: the customer selects items in the mobile app, the restaurant confirms the order, payment is processed, a delivery agent is assigned, and the order is picked up and delivered, with confirmation flowing back to the customer.

8. Conclusion

The Food Delivery Management System connects customers, restaurants, and delivery agents on a single real-time platform, streamlining the entire journey from browsing to doorstep delivery.

User Stories

Captured the real needs of customers, delivery agents, and restaurant admins as the basis for system requirements.

Rapid Application Development

Enabled fast prototyping and iterative refinement of user-facing flows, suited to a competitive, fast-moving market.

Node.js / React Native Stack

Provided a real-time, scalable foundation for live order tracking and high-volume concurrent usage.

Structured Strategy

Ensured systematic design, development, testing, and evaluation, resulting in a reliable platform ready for real-world deployment.